CISS245: Advanced Programming
Quiz q6002

Name: eagraham1@cougars.ccis.edu Score:

Q1. Write a class `Time` that has private members `hh_`, `mm_`, `ss_`, for hours, minutes, seconds in 24-hour format, i.e., hours is in the range 0..23. Write enough public methods so that you can create an array `t` of 10 `Time` objects so that `t[0]` models the time 00:00:00, `t[1]` models the time 00:01:00, `t[2]` models the time 00:02:00, ... `t[9]` models the time 00:09:00. Print all the objects in `t`.

Answer:

```
#include <iostream>

class Time
{
public:
void set_time(int hh, int mm, int ss)
{
  hh_ = hh;
  mm_ = mm;
  ss_ = ss;
}

void println() const
{
  std::cout << std::setw(2) << std::setfill('0') << hh_ << ":"
  << std::setw(2) << std::setfill('0') << mm_ << ":"
  << std::setw(2) << std::setfill('0') << ss_ << std::endl;
}
private:
int hh_;
int mm_;
int ss_;
};

int main()
{
    // Declare t to be an array of 10 Time objects
    Time t[10];
    // In a loop, let t[i] invoke method setHHMMSS(hh, mm, ss)
    // with the right values for hh, mm, ss.
    for (int i = 0; i < 10; i++)
    {
      time[i].set_time(0, i, 0);
    }
```

```
    // In a loop, let t[i] invoke method println() to print
    // the values in t[i] in the format hh:mm:ss, followed by newline.
    // Note that hh:mm:ss must be zero-filled. For instance
    // midnight must be printed by 00:00:00 and not 0:0:0.

    for (int i = 0; i < 10; ++i)
    {
      t[i].println();
    }

    return 0;
}
```

Q2. This is the same as the previous question *except* that `t` is an array of `Time` pointers. You can copy the `Time` class from above to the answer here.
Answer:

```
#include <iostream>

class Time
{
public:
void set_time(int hh, int mm, int ss)
{
  hh_ = hh;
  mm_ = mm;
  ss_ = ss;
}

void println() const
{
  std::cout << std::setw(2) << std::setfill('0') << hh_ << ":"
  << std::setw(2) << std::setfill('0') << mm_ << ":"
  << std::setw(2) << std::setfill('0') << ss_ << std::endl;
}
private:
int hh_;
int mm_;
int ss_;
};

int main()
{
    // Declare t to be an array of 10 Time pointers.
    // In a loop, allocate memory on the heap for t[i] to point to.
    Time * t[10]
    // In a loop, let the object that t[i] points to invoke method
    // setHHMMSS(hh, mm, ss) with the right values for hh, mm, ss.
    for (int i = 0; i < 10; i++)
    {
```

```
      time[i]->set_time(0, i, 0);
    }
    // In a loop, let the object t[i] points to invoke method println()
    // to print the values that the the object t[i] points.
    // Print in the format hh:mm:ss, followed by newline.
    // Note that hh:mm:ss must be zero-filled. For instance
    // midnight must be printed by 00:00:00 and not 0:0:0.
    for (int i = 0; i < 10; ++i)
    {
      t[i]->println();
    }
    // Deallocate memory used by t.
    for (int i = 0; i < 10; i++)
    {
      delete t[i];
    }

    return 0;
}
```

Q3. Wumpus lives in the Wumpus world, which is a 4-by-4 grid. For instance in the following

```
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | |W|
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
```

Wumpus is at row 1, column 3. You are given this:

```
// file: WumpusWorld.h
#ifndef WUMPUSWORLD_H
#define WUMPUSWORLD_H

#include <iostream>

class WumpusWorld
{
public:
    void init();         // initialize so that Wumpus is at (0,0).
    void println();      // print according to the above format
    void move_wumpus();  // randomize a direction rand() % 4 where 0,1,2,3
                         // are N,S,E,W for Wumpus. Of course Wumpus
                         // must stay in the world.
                         // If wumpus tries to move N but its row is 0,
                         // wumpus stays put.
private:
    char world_[4][4];
};
#endif
```

Do not add anything to the above. You want to make Wumpus wonder around like this:

```
#include <iostream>
#include <ctime>
#include <cstdlib>
#include "WumpusWorld.h"

int main()
{
    srand((unsigned int) time(NULL));

    WumpusWorld ww;
    ww.init();
    ww.println();
    for (int i = 0; i < 5; ++i)
    {
        ww.move_wumpus();
        ww.println();
    }
    return 0;
}
```

Complete the following cpp file that contains the implementation of the methods declared in the WumpusWorld class.

Answer:

```
// file: WumpusWorld.cpp

void WumpusWorld::init()
{
```

```
  for (int i = 0; i < 4; ++i)
  {
    for (int j = 0; j < 4; ++j)
    {
      world_[i][j] = ' ';
    }
  }

  world_[0][0] = 'W';
}

void WumpusWorld::println()
{
  for (int i = 0; i < 4; i++)
  {
    std::cout << "+-+-+-+-+" << std::endl;
    for (int j = 0; j < 4; j++)
    {
      std::cout << "|" << world_[i][j];
    }
    std::cout << "|" << std::endl;
  }
  std::cout << "+-+-+-+-+" << std::endl;
}

void WumpusWorld::move_wumpus()
{
  int wumpus_row = -1, wumpus_col = -1;
  for (int i = 0; i < 4; i++)
  {
    for (int j = 0; j < 4; j++)
    {
      if (world_[i][j] == 'W')
      {
        wumpus_row = i;
        wumpus_col = j;
        break;
      }
    }
    if (wumpus_row != -1)
    {
      break;
    }
  }

  int direction = rand() % 4; // 0 - North, 1 - South, 2 - East, 3 - West

  world_[wumpus_row][wumpus_col] = ' ';

  switch (direction)
  {
```

```
    case 0: // North
    if (wumpus_row > 0)
    {
      wumpus_row--;
    }
    break;
    case 1: // South
    if (wumpus_row < 3)
    {
      wumpus_row++;
    }
    break;
    case 2: // East
    if (wumpus_col < 3)
    {
      wumpus_col++;
    }
    break;
    case 3: // West
    if (wumpus_col > 0)
    {
      wumpus_col--;
    }
    break;
  }

  world_[wumpus_row][wumpus_col] = 'W';
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.